PROGRAMOWANIE OBIEKTOWE W C++

Część 2: Klasy, Enkapsulacja i Konstruktory

Prowadzący: dr Marcin Misiaszek

Rok akademicki 2025/2026

1. Dlaczego Programowanie Obiektowe (OOP)?

Programowanie proceduralne (C) skupia się na **funkcjach** operujących na danych. OOP skupia się na **obiektach**, które łączą dane i zachowania.

Trzy filary OOP:

- **Enkapsulacja (Hermetyzacja)** – Ukrywanie stanu wewnętrznego i udostępnianie interfejsu.
- **Dziedziczenie** – Tworzenie nowych klas na bazie istniejących.
- **Polimorfizm** – Jednolity interfejs dla różnych typów danych.

> **Obiekt = Stan (Dane) + Zachowanie (Metody)**

2. Od Struktur do Klas

W C++

```
struct
```

i

```
class
```

są niemal identyczne. Różnią się jedynie **domyślnym poziomem dostępu**.

Struktura (struct)

- Domyślnie: public
- Często używana dla prostych typów danych z domyślnie publicznymi polami.

Klasa (class)

- Domyślnie: private
- Używana, gdy chcemy chronić dane i wymusić interfejs.

```
class Date {
    int m_year; // Prywatne domyślnie!
public:
    void print(); // Publiczne
};
```

3. Specyfikatory Dostępu: public vs private

Kontrola nad tym, kto może modyfikować wnętrze naszego obiektu.

- **private:** Dostępne tylko dla metod wewnątrz tej samej klasy. Chroni przed błędami logicznymi z zewnątrz.
- **public:** Interfejs klasy. Widoczny dla każdego, kto posiada obiekt.

> **Złota zasada: W większości klas pola danych (zmienne członkowskie) powinny być prywatne.**

```
private:
    int m_month { 1 }; // m_ oznacza "member" (pole klasy)
```

4. Interfejs publiczny klasy

Dostęp do danych powinien odbywać się przez publiczny interfejs klasy, często w postaci getterów, setterów lub metod operujących na stanie obiektu.

Zalety stosowania metod zamiast bezpośredniego dostępu:

- **Walidacja:** Metoda może sprawdzić poprawność wprowadzanych danych.
- **Kontrola:** Możemy udostępnić tylko odczyt lub specyficzne operacje domenowe.
- **Abstrakcja:** Możemy zmienić wewnętrzną reprezentację danych bez zmiany kodu użytkownika.

```
int getMonth() const { return m_month; } // Getter
void setMonth(int m) {
    if (m >= 1 && m <= 12) m_month = m; // Walidacja!
}
```

5. Metody Pomocnicze i Spójność Stanu

Nie każda metoda musi być publiczna. Metody prywatne pomagają organizować kod wewnątrz klasy.

Utrzymywanie inwariantów (spójności):

Obiekt nie powinien nigdy wejść w stan nielogiczny.

```
class Employee {
    std::string m_name;
    char m_initial;

public:
    void setName(std::string_view name) {
        m_name = name;
        updateInitial(); // Wariant projektowy: metoda pomocnicza
    }
private:
    void updateInitial() { m_initial = m_name[0]; }
};
```

6. Konstruktory: Narodziny Obiektu

Konstruktor to specjalna funkcja składowa klasy wywoływana automatycznie w momencie tworzenia obiektu.

- Ma taką samą nazwę jak klasa.
- **Nie posiada** typu zwracanego (nawet void).
- Może być przeciążany (wiele konstruktorów dla jednej klasy).

```
class Foo {
public:
    Foo() { /* domyślny */ }
    Foo(int x, int y) { /* parametryczny */ }
};

Foo f1;        // Woła domyślny
Foo f2{ 5, 10 }; // Woła parametryczny
```

7. Lista Inicjalizacyjna Członków

Najważniejszy mechanizm inicjalizacji w C++. Wykonywany **zanim** wejdziemy do klamerek konstruktora.

> **Pozwala zainicjalizować pola bezpośrednio, zamiast najpierw tworzyć je w stanie początkowym, a potem przypisywać im nowe wartości.**

```
// Przypisanie (mniej efektywne dla typów złożonych):
Foo(int x) { m_x = x; }

// Inicjalizacja (zalecana):
Foo(int x) : m_x{ x } // Lista inicjalizacyjna
{
}
```

Wymagane dla: pól

```
const
```

, referencji oraz klas bez konstruktora domyślnego.

8. Dobre Praktyki (Best Practices)

- **Const Correctness:** Każda metoda, która tylko czyta dane, powinna mieć

  ```
  const
  ```

  na końcu.

- **Nazewnictwo:** Używaj

  ```
  m_
  ```

  dla pól prywatnych, aby odróżnić je od parametrów.

- **Single Responsibility:** Klasa powinna odpowiadać za jedną logiczną spójną rzecz.
- **std::string_view:** Wydajny sposób przekazywania tekstów bez niepotrzebnych kopii.

ANALIZA PROGRAMÓW ĆWICZENIOWYCH

Ewolucja kodu: Od struktury do klasy

Program cw2-1.cpp: Podejście proceduralne

Punkt wyjścia: klasyczna struktura w stylu C, używana jako zwykły kontener na dane (Plain Old Data).

Zastosowane techniki:

- **Agregat danych:** Definicja

  ```
  struct Date
  ```

  z polami inicjalizowanymi wartością zero za pomocą

  ```
  {}
  ```

  .

- **Domyślny dostęp:** W

  ```
  struct
  ```

  pola są domyślnie

  ```
  public
  ```

  .

- **Referencja do const:** Wydajne przekazywanie obiektu do funkcji bez kopiowania.

Program cw2-2.cpp: Pierwsza metoda

Pierwszy krok w stronę obiektowości – połączenie danych i zachowania wewnątrz jednego typu.

Zastosowane techniki:

- **Metoda członkowska:** Definicja

  ```
  print()
  ```

  wewnątrz struktury.
- **Różnica C vs C++:** W C++ struktury mogą zawierać funkcje, w czystym C struktury przechowują wyłącznie dane.
- **Operator kropki:** Jednolita składnia wywołania:

  ```
  today.print()
  ```

  .

Program cw2-3.cpp: Enkapsulacja i Klasy

Wprowadzenie ścisłej kontroli dostępu i świadomego projektowania interfejsu.

Zastosowane techniki:

- **class vs struct:** Zmiana słowa kluczowego wymusza domyślną prywatność (

  ```
  private
  ```

).
- **In-class member initializers:** Bezpieczne wartości domyślne przy deklaracji pól.
- **Metoda const:** Gwarancja, że metoda nie modyfikuje stanu obiektu (tylko odczyt).

Program cw2-4.cpp: Kontrola dostępu (1/2)

Pełny model klasy wykorzystujący akcesory i mutatory do zarządzania stanem.

Zastosowane techniki:

- **Gettery i Settery:** Publiczne metody do kontrolowanego odczytu i zapisu prywatnych danych.
- **Zalety abstrakcji:** Ukrycie wewnętrznej reprezentacji danych przed użytkownikiem klasy.

Program cw2-4.cpp: Kontrola dostępu (2/2)

Dalsza analiza:

- **Punkt walidacji:** Settery umożliwiają dodanie logiki sprawdzającej poprawność danych (np. zakres miesiąca 1-12).
- **Wniosek:** Publiczny interfejs tworzy bezpieczny most między prywatnym wnętrzem a światem zewnętrznym.

Program cw2-5.cpp: Spójność wewnętrzna (1/2)

Przykład klasy dbającej o inwarianty – logiczne powiązania między polami.

Zastosowane techniki:

- **Zależności danych:** Metoda

  ```
  setName()
  ```

  aktualizuje jednocześnie imię i inicjał.
- **std::string_view:** Nowoczesny typ (C++17) zapewniający wydajny dostęp do tekstu.

Program cw2-5.cpp: Spójność wewnętrzna (2/2)

Dalsza analiza:

- **Zapewnienie inwariantu:** Klasa uniemożliwia powstanie stanu niespójnego.
- **Uwaga:** Wywołanie

  ```
  name.front()
  ```

  zakłada, że przekazany napis nie jest pusty.

Program cw2-6.cpp: Cykl życia - Konstruktor (1/2)

Zrozumienie mechanizmu automatycznego powoływania obiektów do życia.

Zastosowane techniki:

- **Definicja konstruktora:** Specjalna funkcja o nazwie klasy wywoływana przy tworzeniu obiektu.
- **Konstruktor parametryczny:** Możliwość przekazywania wartości podczas inicjalizacji.

Program cw2-6.cpp: Cykl życia - Konstruktor (2/2)

Dalsza analiza:

- **Pułapka inicjalizacji:** Argumenty konstruktora to nie to samo co pola obiektu (wymagają jawnego przypisania).
- **Lekcja:** Konstruktor bez implementacji pozostawia pola w ich stanie domyślnym.

Program cw2-7.cpp: Inicjalizacja (1/2)

Najbardziej efektywny sposób inicjalizacji członków klasy w nowoczesnym C++.

Zastosowane techniki:

- **Member Initializer List:** Składnia

  ```
  : m_x{x}
  ```

  przed ciałem konstruktora.
- **Wydajność:** Bezpośrednia inicjalizacja pola bez etapu domyślnego tworzenia.

Program cw2-7.cpp: Inicjalizacja (2/2)

Dalsza analiza:

- **Wymogi techniczne:** Niektóre pola (

  ```
  const
  ```

  , referencje) MUSZĄ być inicjalizowane na liście.
- **Wniosek:** Lista inicjalizacyjna to standard zapewniający poprawność i wydajność.

9. Podsumowanie Wykładu

1. **Klasa** łączy dane i metody, tworząc spójny obiekt.
2. W klasach dane są zwykle ukrywane, a dostęp do nich odbywa się przez **publiczny interfejs**.
3. Metody **const** gwarantują bezpieczeństwo odczytu stanu obiektu.
4. **Konstruktor** automatyzuje inicjalizację, a **Lista Inicjalizacyjna** jest preferowanym sposobem jego implementacji.

Zadania do wykonania na ćwiczeniach

- Zaimplementuj klasę

  ```
  Rectangle
  ```

  z prywatnymi polami

  ```
  m_width
  ```

  i

  ```
  m_height
  ```

  .

- Stwórz konstruktor wykorzystujący **listę inicjalizacyjną**.
- Dodaj settery z **walidacją** (np. szerokość musi być większa od zera).
- Dodaj metodę pomocniczą

  ```
  double calculateArea() const
  ```

  .

- W

  ```
  main()
  ```

  przetestuj działanie walidacji oraz metod odczytu.